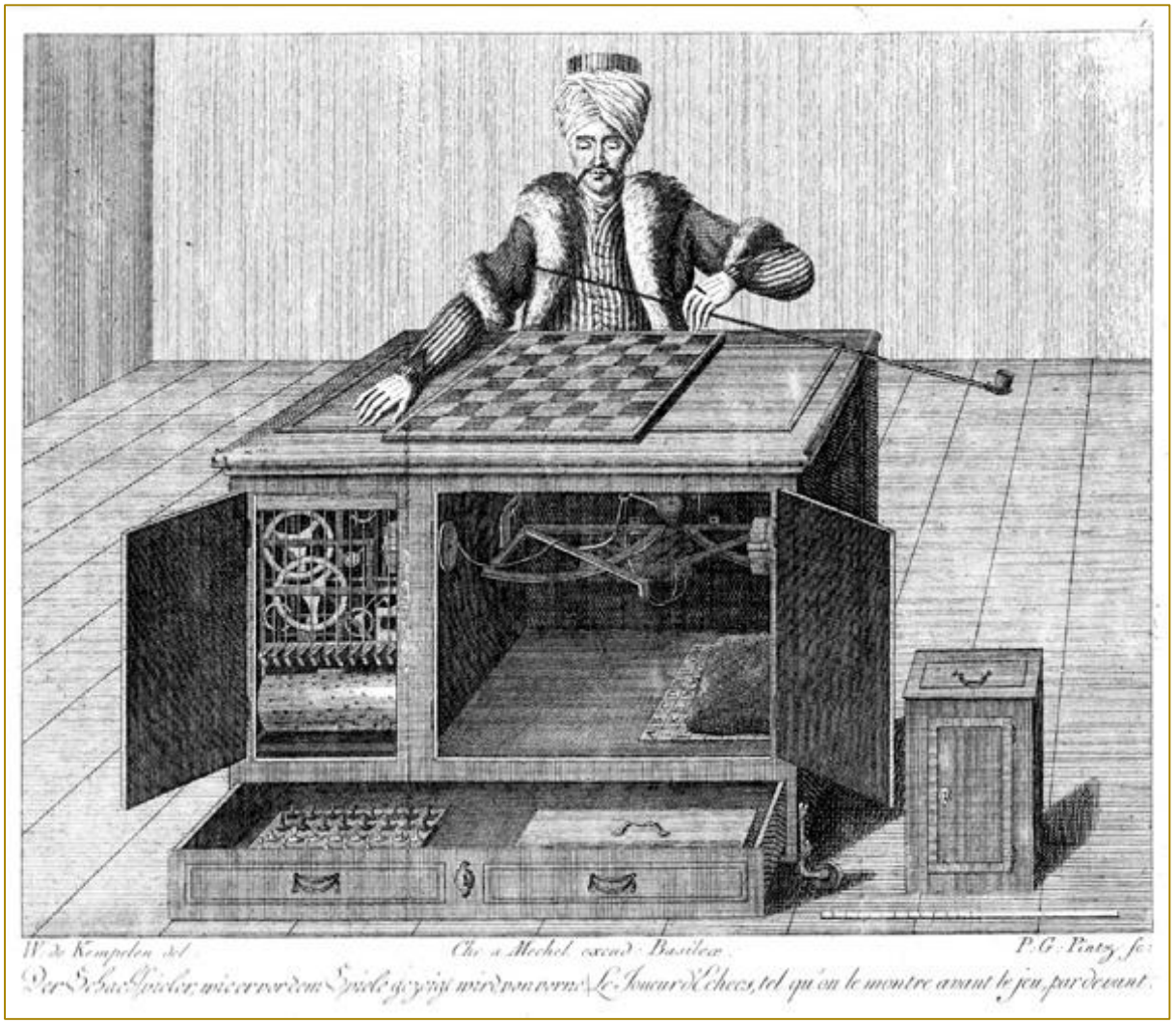
CS 380: ARTIFICIAL INTELLIGENCE

Lecture 6: Adversarial Search I

Recall: Problem Solving

- Idea: represent the problem we want to solve as:
  - State space
  - Actions
  - Goal check
  - Cost function
- If this is possible, we can use search methods to solve any problem:
  - Systematic Search:
    - BFS, DFS, ID, Greedy Search, A*
  - Local Search:
    - Hill Climbing, Simulated Annealing, Local Beam Search (e.g., GAs)

The “Mechanical Turk” (1770-1854)



Wikipedia.org

Games vs. Search Problems

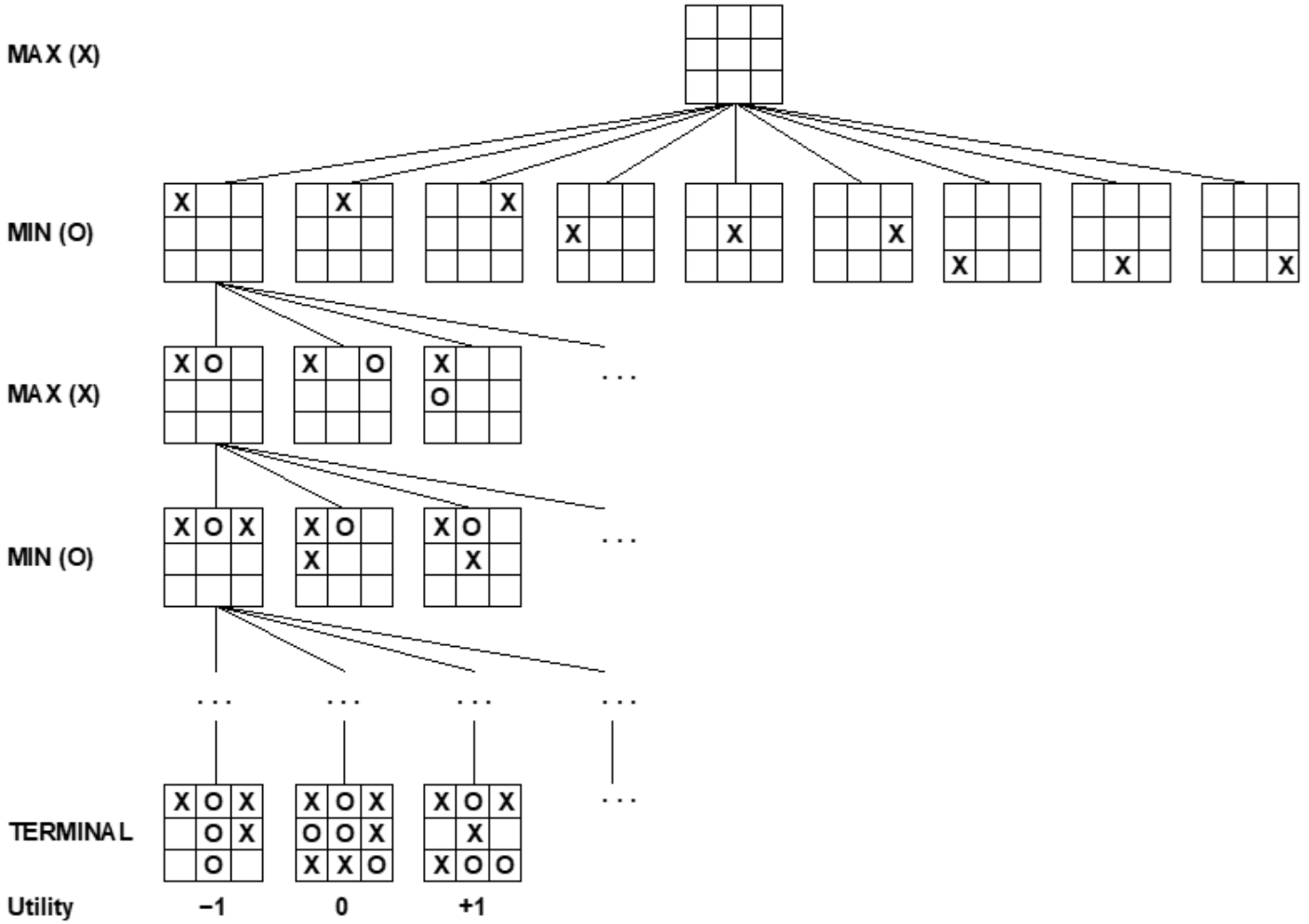
- "Unpredictable" opponent => solution is a strategy specifying a move for every possible opponent reply
- Time limits => unlikely to find goal, must approximate
- Plan of Attach:
  - Computer considers possible lines of play (Babbage, 1846)
  - Algorithm for perfect play (Zermelo, 1912; Von Neumann, 1944)
  - Finite horizon, approximate evaluation (Zuse, 1945; Wiener, 1948; Shannon, 1950)
  - First chess program (Turing, 1951)
  - Machine learning to improve evaluation accuracy (Samuel, 1952-57)
  - Pruning to allow deeper search (McCarthy, 1956)

Types of Games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships, blind tictactoe	bridge, poker, scrabble

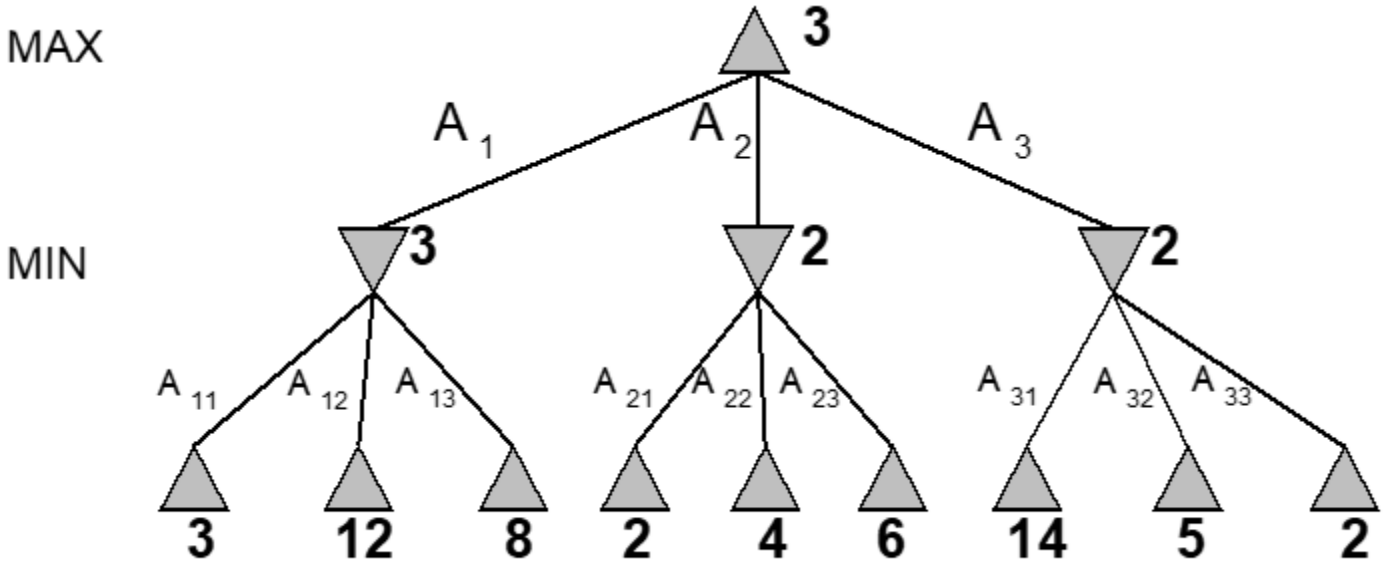
Players make a move simultaneously; if they select the same spot, coin flip determines mark

Game Tree (2-player, deterministic, turns)

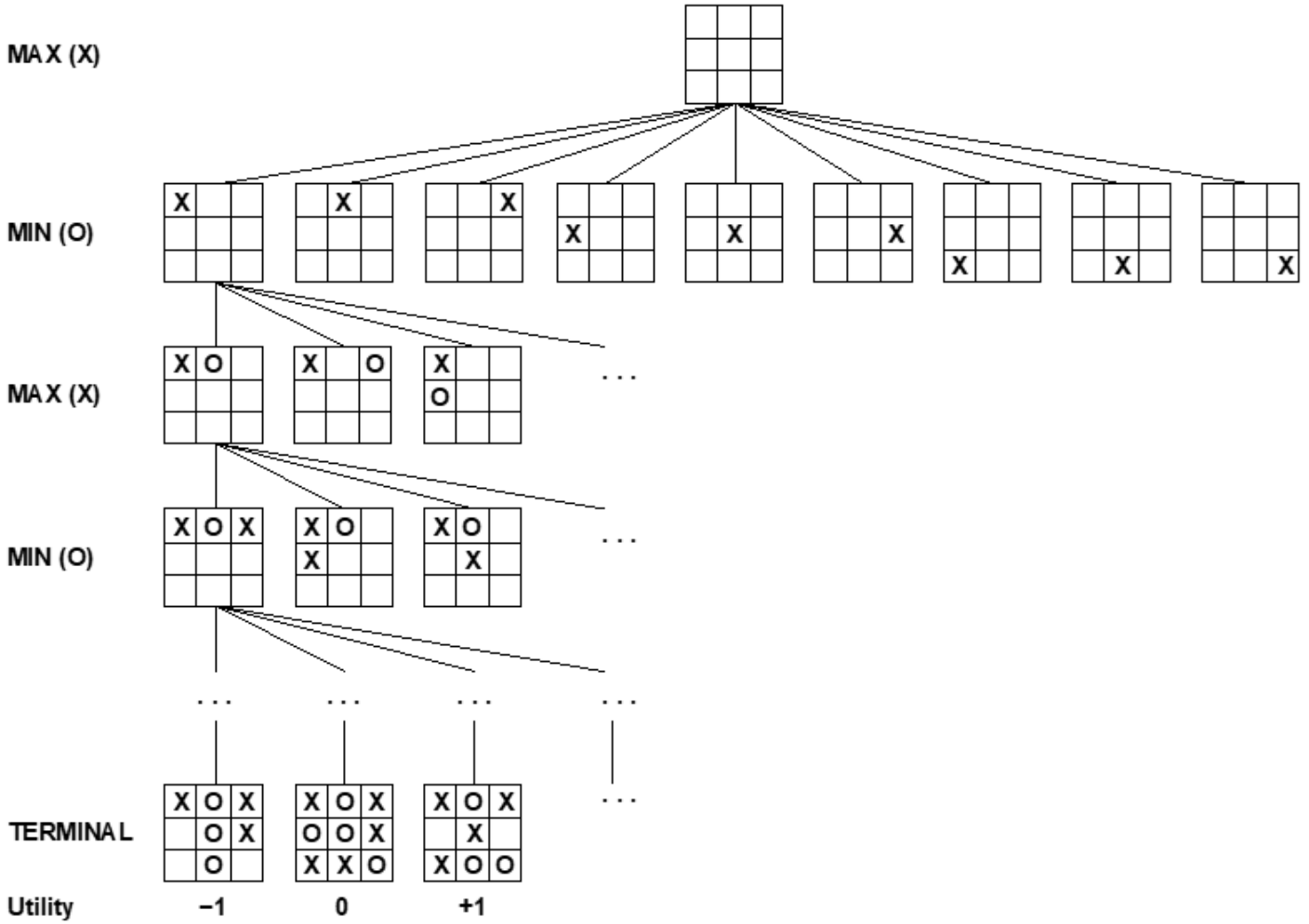


Minimax

- Perfect play for deterministic, perfect-information games
- Idea: choose move to position with highest minimax value = best achievable payoff against best play
- E.g., 2-ply game:



Game Tree (2-player, deterministic, turns)



Minimax Algorithm

```
function Minimax-Decision(state) returns an action
    inputs: state, current state in game
    return the a in Actions(state) maximizing Min-Value(Result(a, state))

function Max-Value(state) returns a utility value
    if Terminal-Test(state) then return Utility(state)
    v ← −∞
    for a, s in Successors(state) do v ← Max(v, Min-Value(s))
    return v

function Min-Value(state) returns a utility value
    if Terminal-Test(state) then return Utility(state)
    v ← ∞
    for a, s in Successors(state) do v ← Min(v, Max-Value(s))
    return v
```

Properties of Minimax

- Complete ??
- Optimal ??
- Time Complexity ??
- Space Complexity ??

Properties of Minimax

- Complete : Only if tree is finite (chess has specific rules for this) NB a finite strategy can exist even in an infinite tree!
- Optimal ??
- Time Complexity ??
- Space Complexity ??

Properties of Minimax

- Complete : Yes, if tree is finite
- Optimal : Yes, against an optimal opponent. Otherwise??
- Time Complexity ??
- Space Complexity ??

Properties of Minimax

- Complete : Yes, if tree is finite
- Optimal : Yes, against an optimal opponent. Otherwise??
- Time Complexity : $O(b^m)$
- Space Complexity ??

Properties of Minimax

- Complete : Yes, if tree is finite
- Optimal : Yes, against an optimal opponent. Otherwise??
- Time Complexity : $O(b^m)$
- Space Complexity : $O(bm)$ (depth-first exploration)

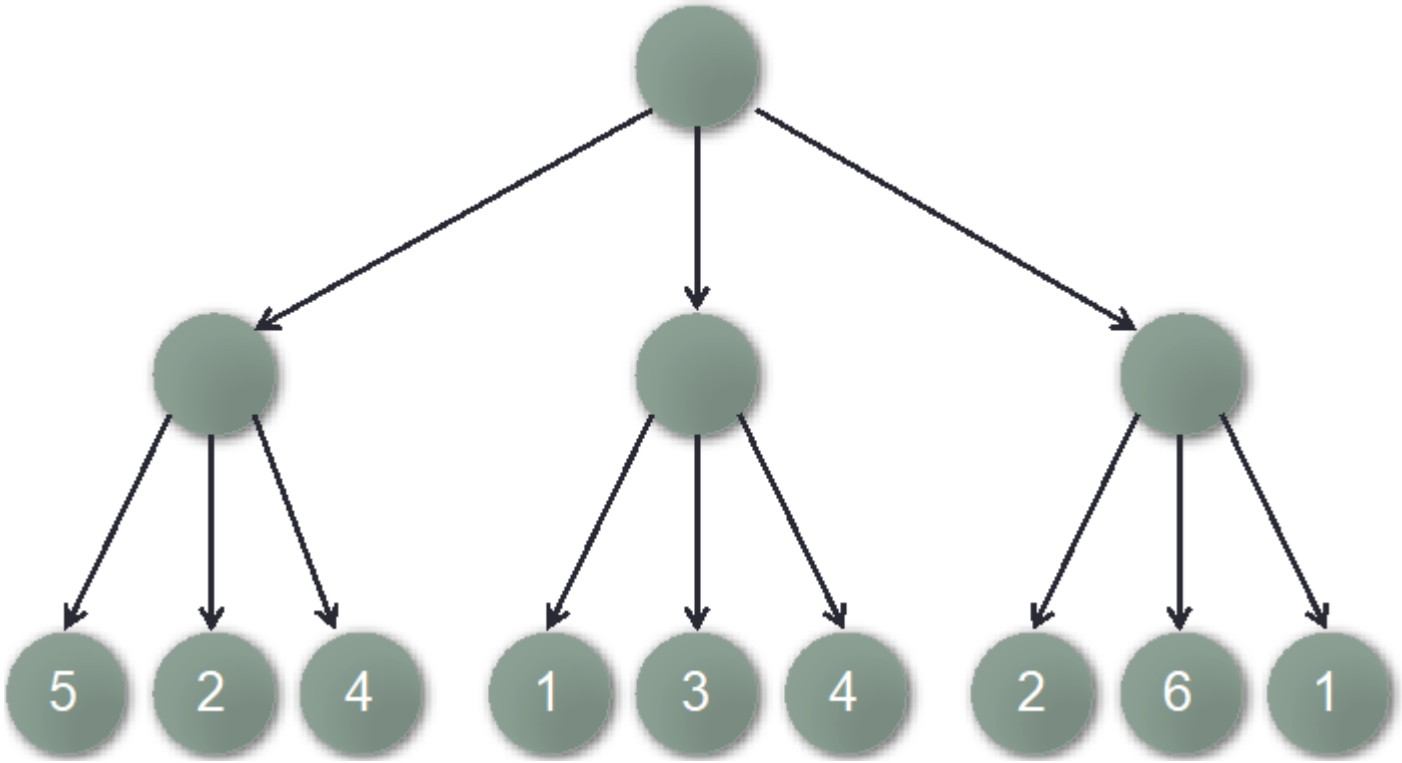
For chess, b ≈ 35, m ≈ 100 for "reasonable" games

=> exact solution completely infeasible

But do we need to explore every path?

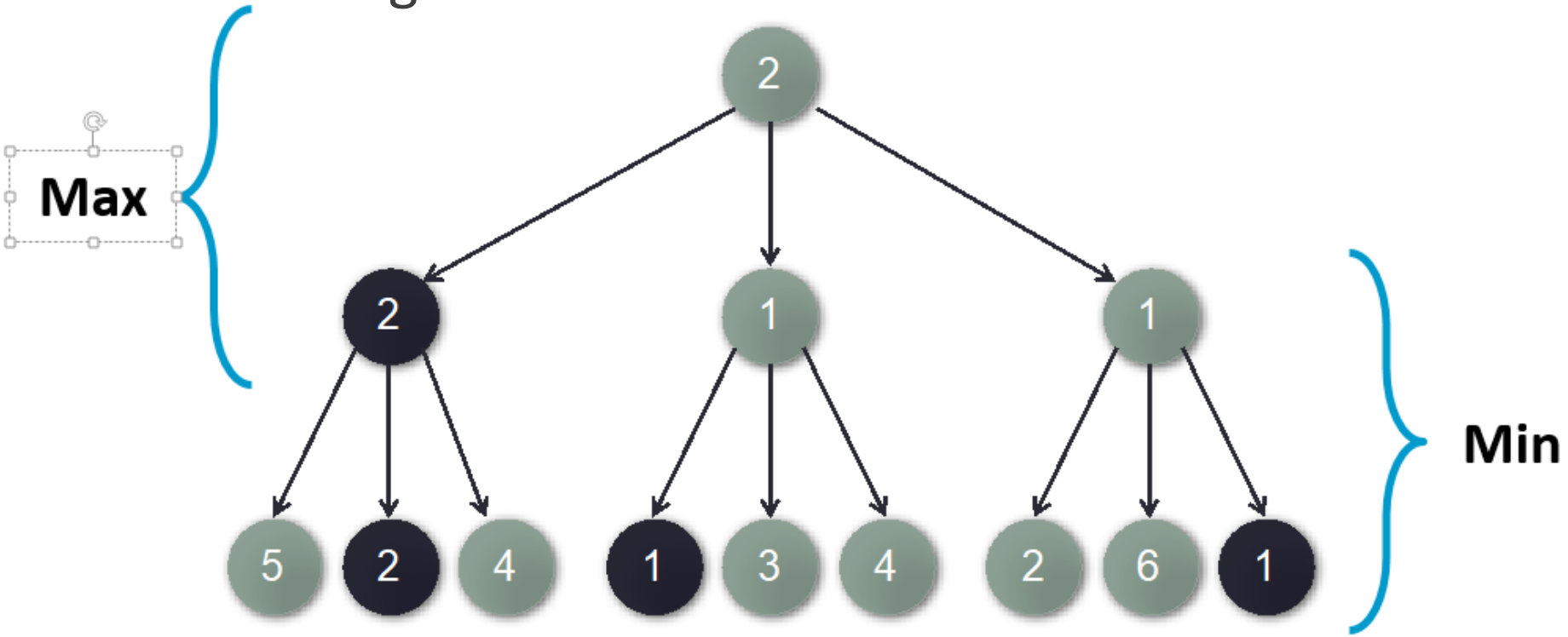
Alpha-Beta Pruning

- Not all nodes in the search tree are relevant for deciding the next move



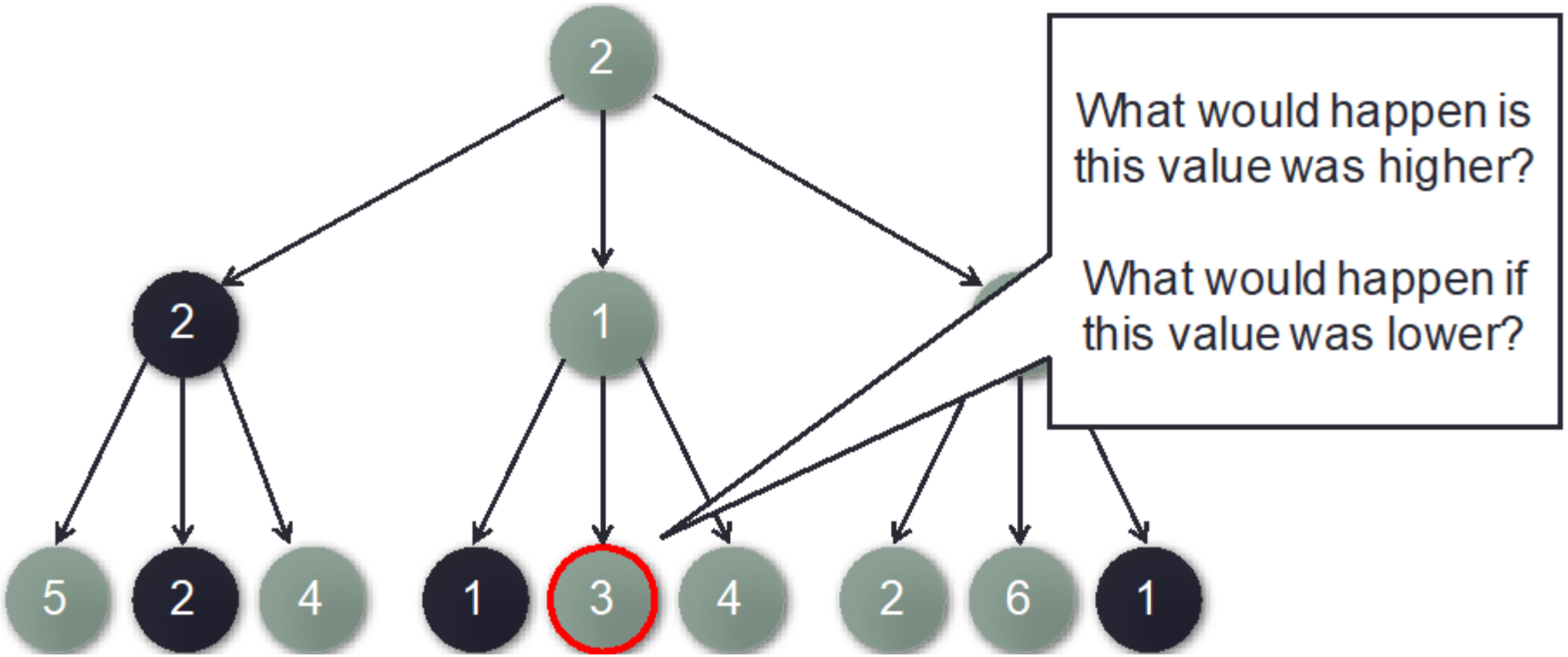
Alpha-Beta Pruning

- Not all nodes in the search tree are relevant for deciding the next move



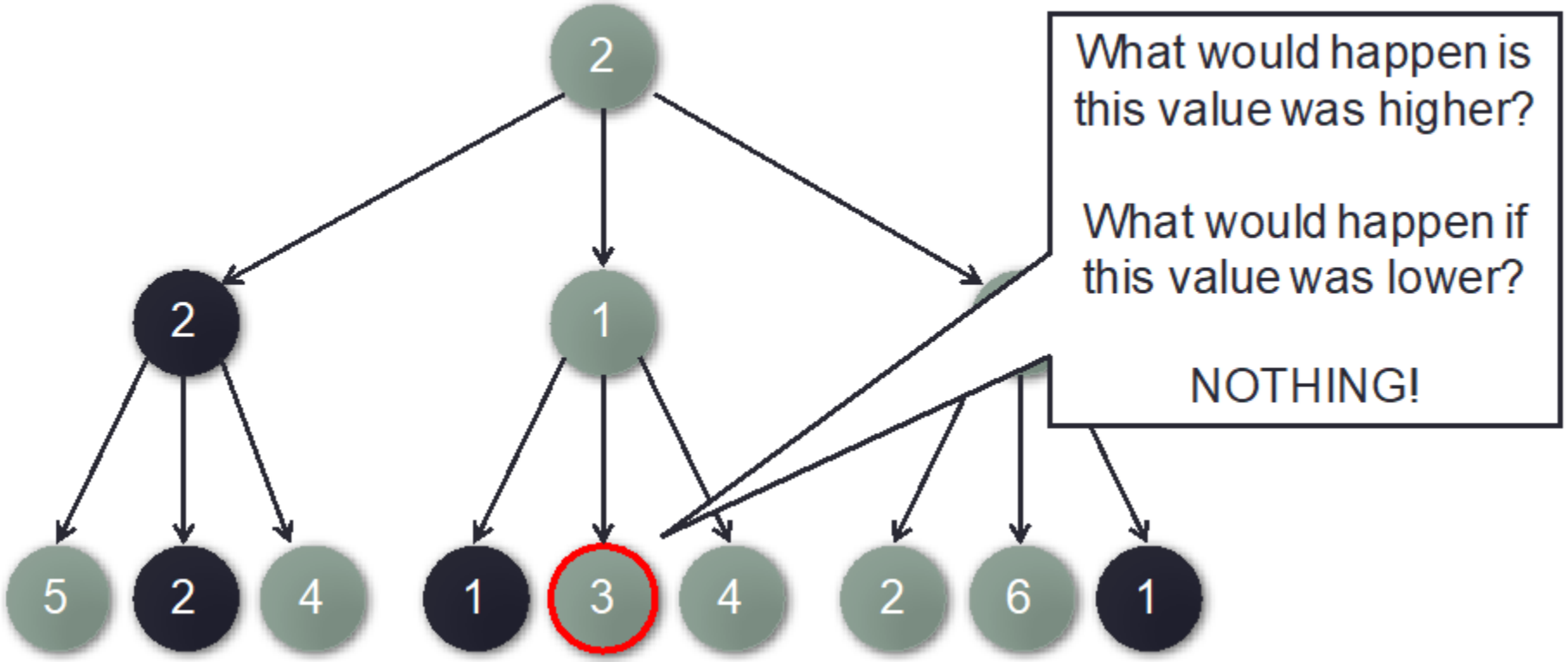
Alpha-Beta Pruning

- Not all nodes in the search tree are relevant for deciding the next move

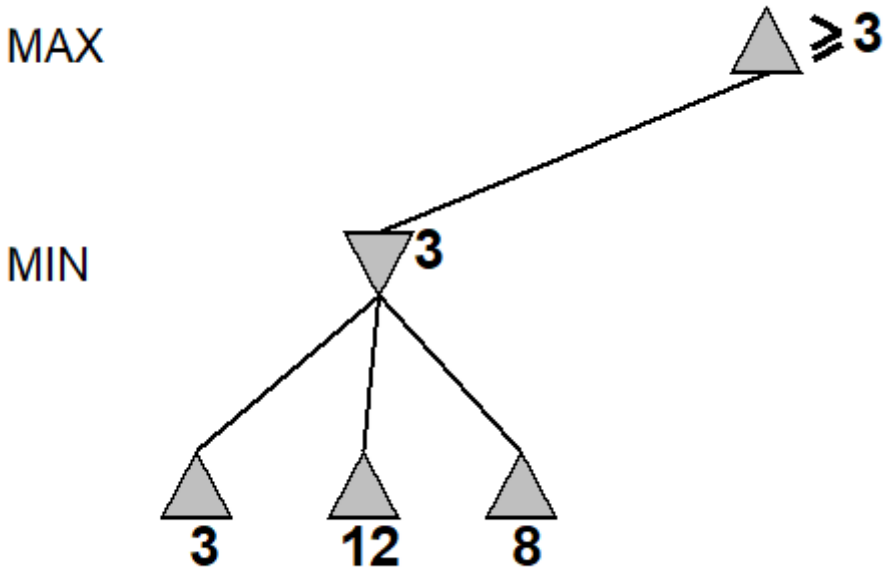


Alpha-Beta Pruning

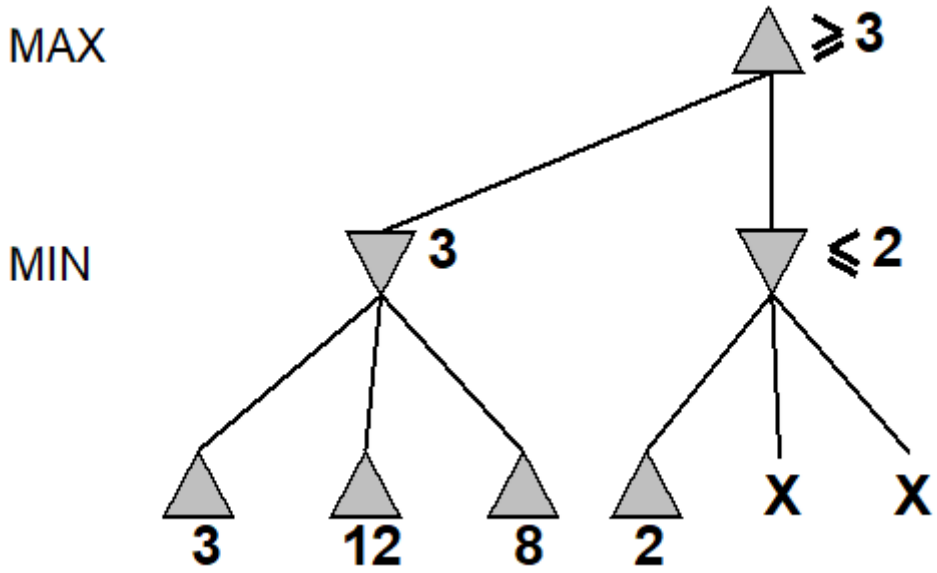
- Not all nodes in the search tree are relevant for deciding the next move



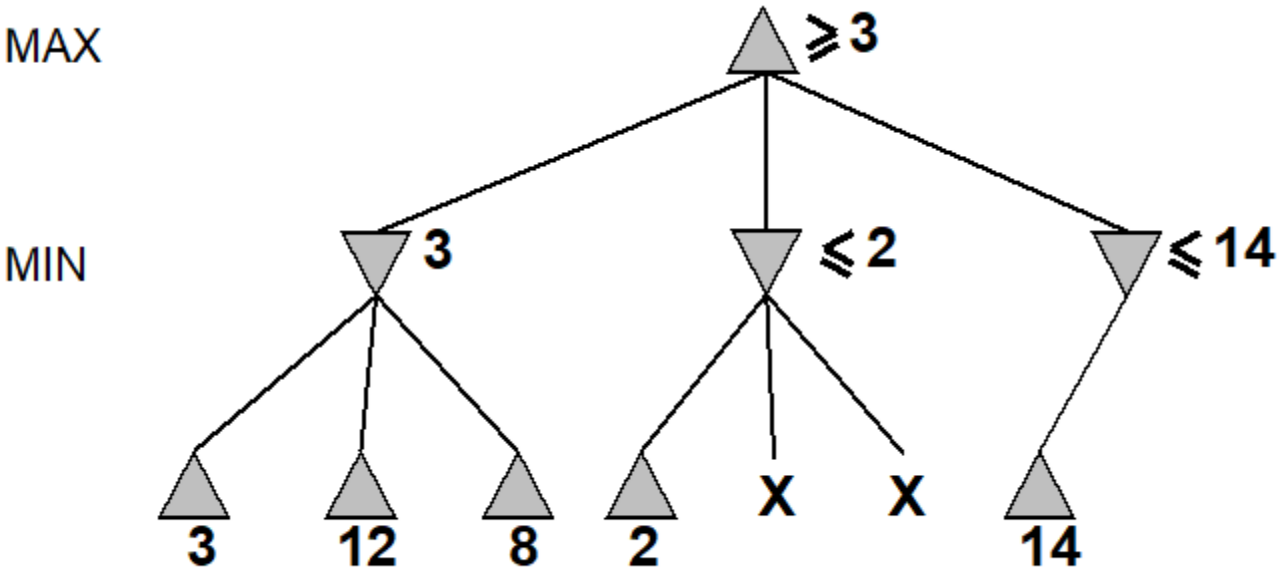
α-β Pruning Example



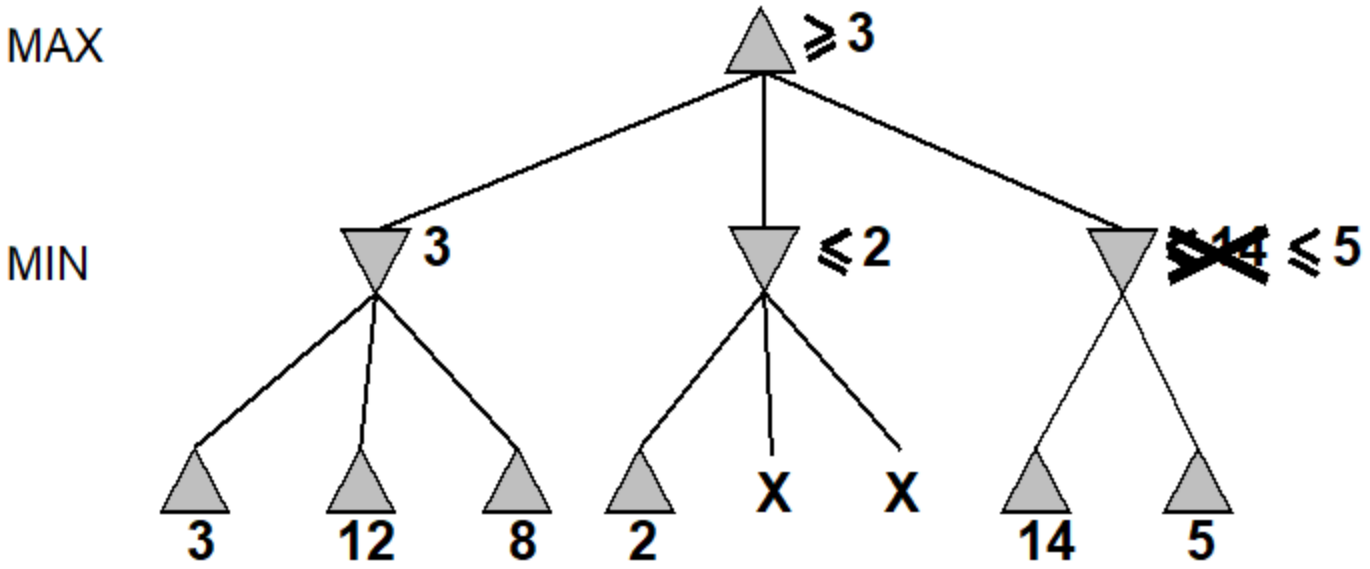
α-β Pruning Example



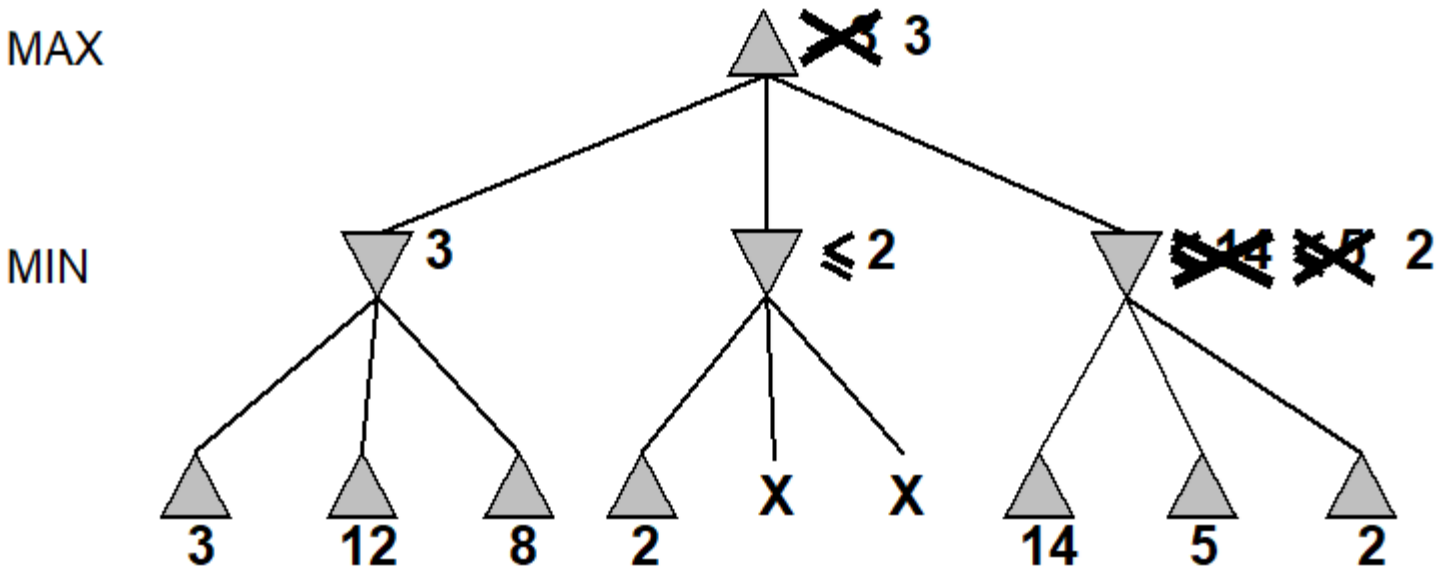
α-β Pruning Example



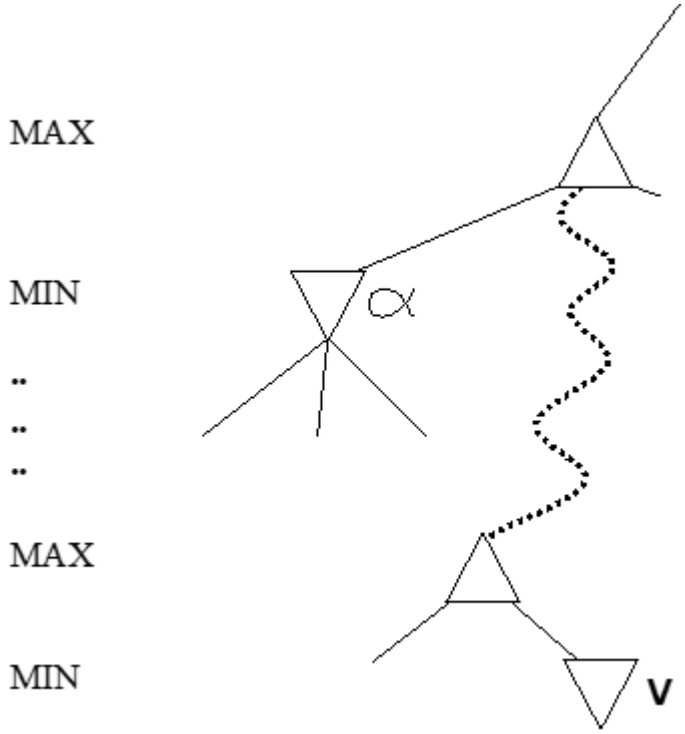
α-β Pruning Example



α-β Pruning Example



Why is it called α-β?



α is the best value (to max) found so far off the current path
If V is worse than α, max will avoid it => prune that branch
Define β similarly for min

The α-β Algorithm

```
function Alpha-Beta-Decision(state) returns an action
    return the a in Actions(state) maximizing Min-Value(Result(a, state))

---

function Max-Value(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for max along the path to state
            β, the value of the best alternative for min along the path to state

    if Terminal-Test(state) then return Utility(state)
    v ← −∞
    for a, s in Successors(state) do
        v ← Max(v, Min-Value(s, α, β))
        if v ≥ β then return v
        α ← Max(α, v)
    return v

---

function Min-Value(state, α, β) returns a utility value
    same as Max-Value but with roles of α, β reversed
```

Properties of α-β

- Pruning does not affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering", time complexity = $O(b^{m/2})$ => doubles solvable depth
- A simple example of the value of reasoning about which computations are relevant (a form of metareasoning)
- Unfortunately, 35^{50} is still impossible!

Resource Limits

- Standard approach:
  - Use Cutoff-Test instead of Terminal-Test
    - e.g., depth limit (perhaps add quiescence search)
  - Use Eval instead of Utility
    - e.g., evaluation function that estimates desirability of position

I.e., search "interesting" states more than "standard" ones

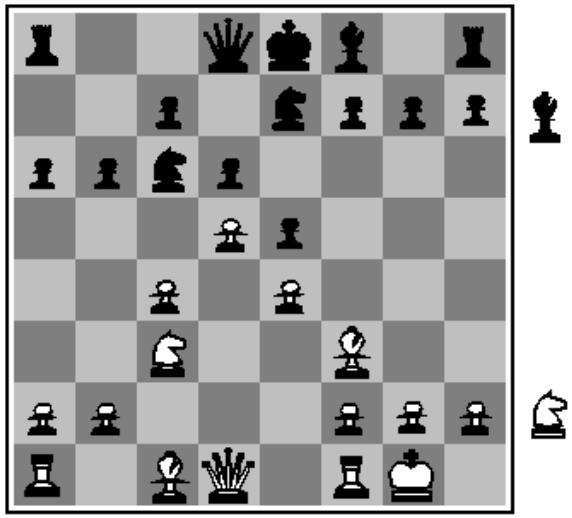
Utility for terminal states; Eval for ALL states

- Suppose we have 100 seconds, explore 10^4 nodes/second

  => 10^6 nodes per move ≈ $35^{8/2}$

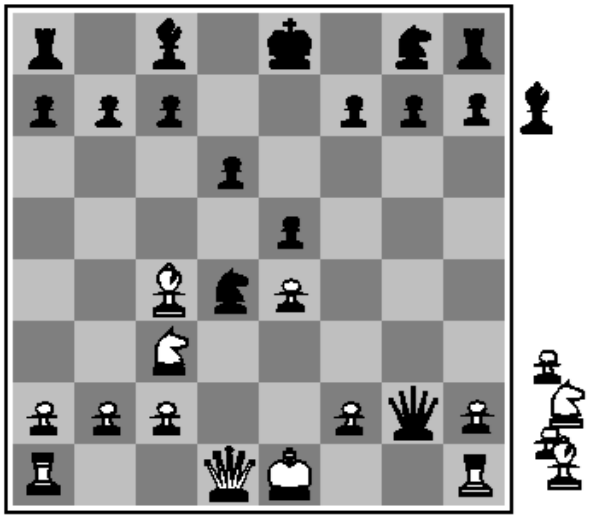
  => α-β reaches depth 8 => pretty good chess program

Evaluation Functions



Black to move

White slightly better



White to move

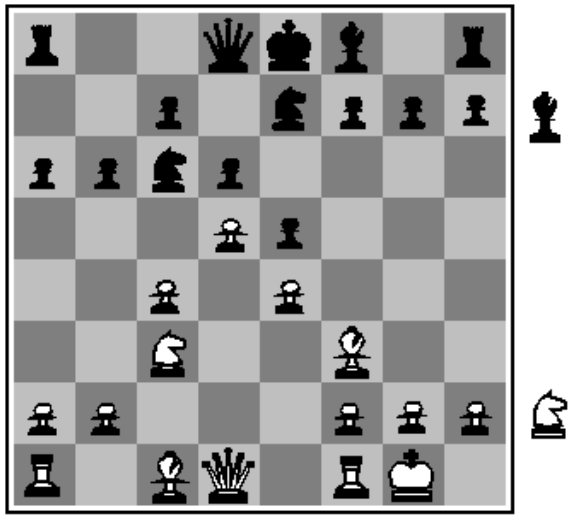
Black winning

- For chess, typically linear weighted sum of features

$$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

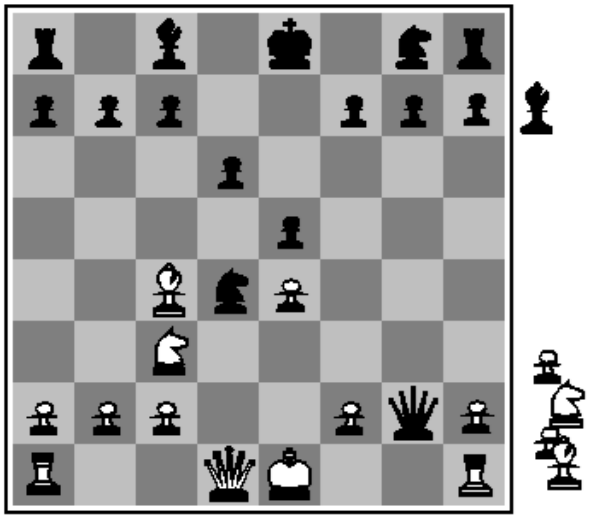
e.g., $w_1 = 9$ with $f_1(s)$ = (number of white queens) – (number of black queens), etc.

Evaluation Functions



Black to move

White slightly better



- For chess, typically linear

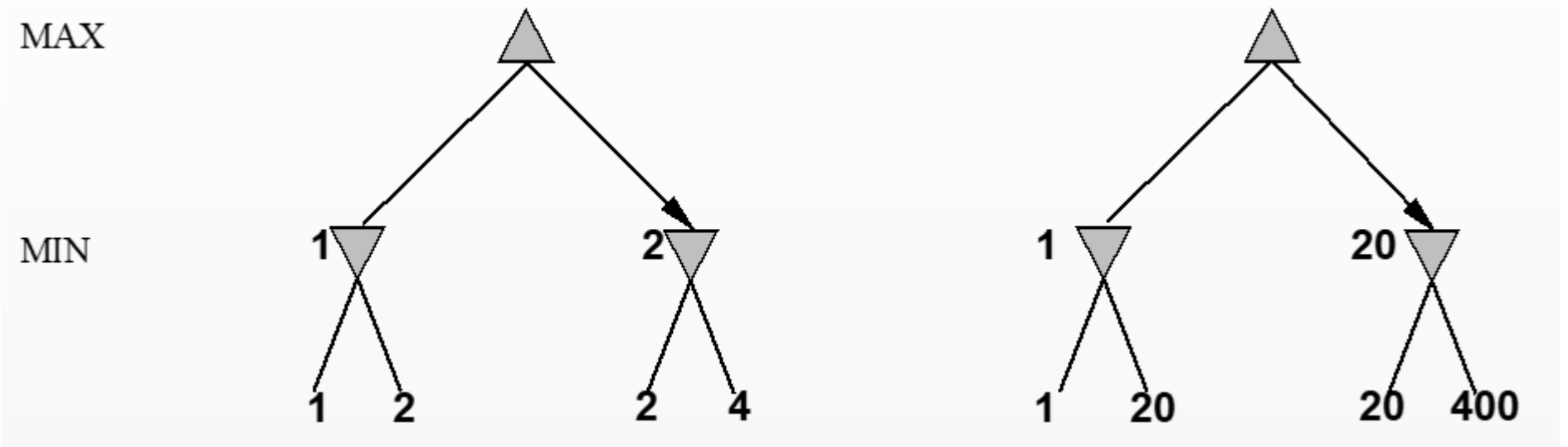
$$Eval(s) = w_1 f_1(s) + w_2$$

e.g., $w_1 = 9$ with $f_1(s) =$ (number c
queens), etc.

Symbol	♙	♘	♗	♖	♕
Piece	pawn	knight	bishop	rook	queen
Value	1	3	3	5	9

3	3.25	5	9	Fischer
3	3.15	4.5	9	Kasparov

Digression: Exact values don't matter



- Behavior is preserved under any monotonic transformation of Eval
- Only the order matters:
  - Payoff in deterministic games acts as an ordinal utility function

Deterministic games in practice

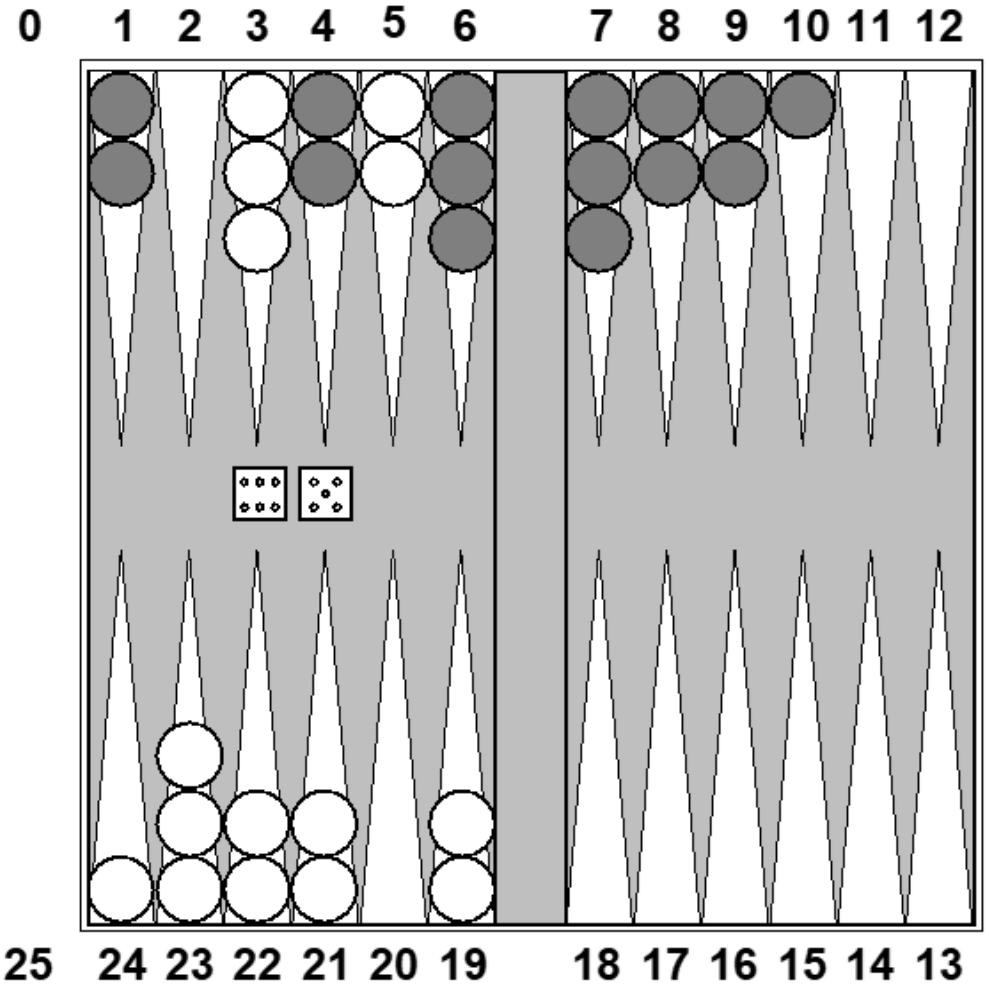
- Checkers: Chinook ended 40-year reign of human world champion Marion Tinsley in 1994. Used an endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 443,748,401,247 positions.
- Chess: Deep Blue defeated human world champion, Gary Kasparov, in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.
- Othello: human champions refuse to compete against computers, who are too good.
- Go: human champions refuse to compete against computers, who are too bad. In go, b > 300, so most programs use pattern knowledge bases to suggest plausible moves.

Deterministic games in practice

- Checkers: Chinook ended 40-year reign of human world champion Marion Tinsley in 1994. Used an endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 443,748,401,247 positions.
- Chess: Deep Blue defeated human world champion, Gary Kasparov, in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.
- Othello: human champions refuse to compete against computers, who are too good.
- Go: human champions refuse to compete against computers, who are too bad. In go, b > 300, so most programs use pattern knowledge bases to suggest plausible moves.

AlphaGo

Nondeterministic games: backgammon

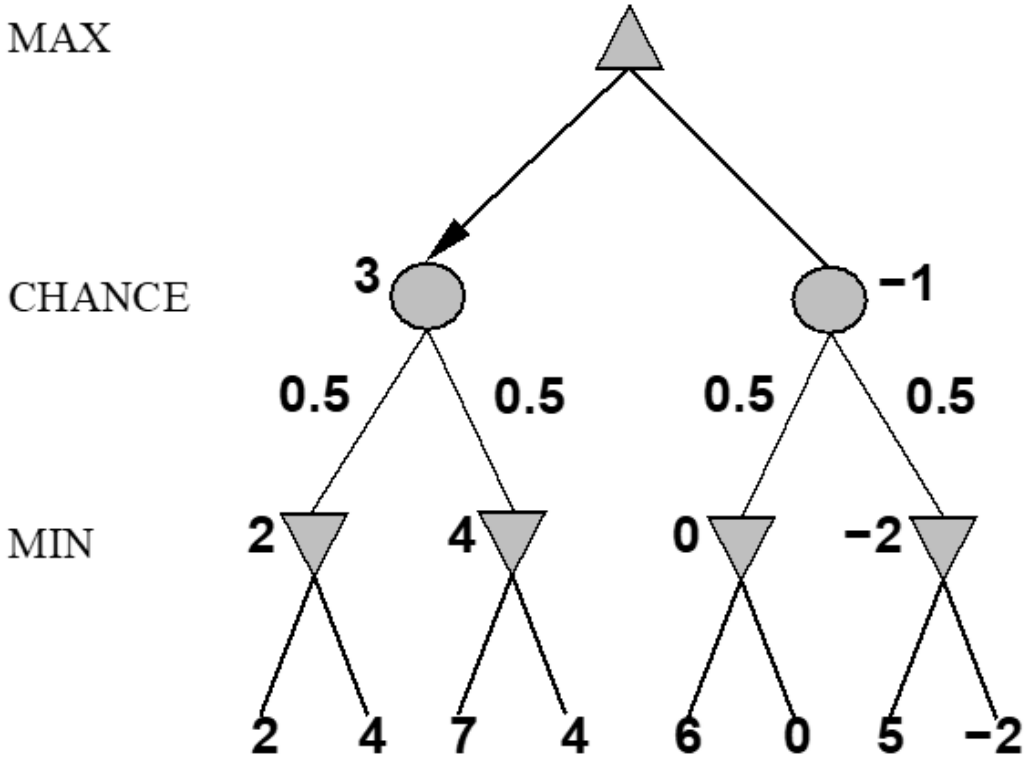


Expecti-minimax

- For chance layer, compute a score weighted by the probability that a move occurs
- In other words: Use the expected value of a move, (instead of max or min) weighted by the probability of that branch of the tree

Nondeterministic games in general

- In nondeterministic games, chance introduced by dice, card-shuffling
- Simplified example with coin-flipping



Algorithm for nondeterministic games

- Expecti-minimax gives perfect play
- Just like minimax, except we must also handle chance nodes

```
...
if state is a Max node then
        return the highest ExpectiMinimax-Value of Successors(state)
if state is a Min node then
        return the lowest ExpectiMinimax-Value of Successors(state)
if state is a chance node then
        return average of ExpectiMinimax-Value of Successors(state)
...
```

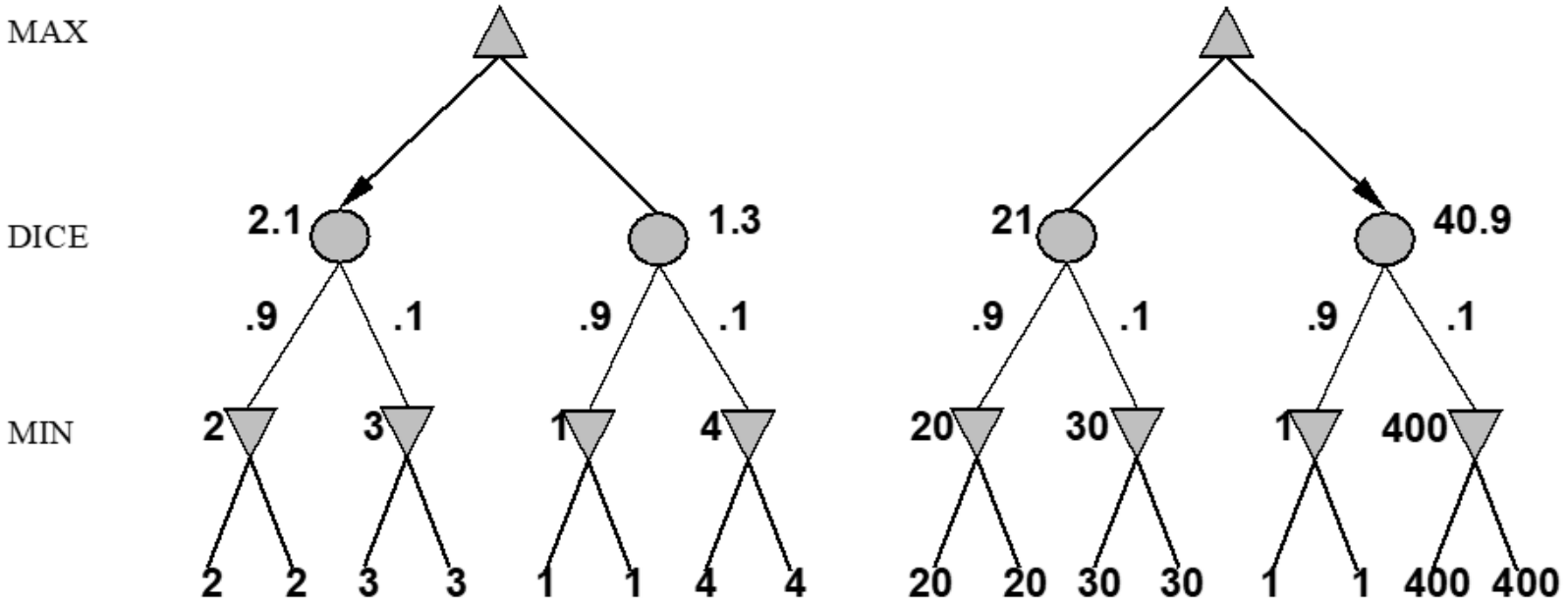
Nondeterministic games in practice

- Dice rolls increase b: 21 possible rolls with 2 dice
- Backgammon ≈ 20 legal moves

  Depth 4 = $20 \times (21 \times 20)^3 \approx 1.2 \times 10^9$

- As depth increases, probability of reaching a given node shrinks => value of lookahead is diminished
- α-β pruning is much less effective
- TDGammon uses depth-2 search + very good Eval
  ≈ world-champion level

Digression: Exact values DO matter



- Behavior is preserved only by positive linear transformation of Eval
- Hence Eval should be proportional to the expected payoff

Games of Imperfect Information

- E.g., card games, where opponent's initial cards are unknown
- Typically, we can calculate a probability for each possible deal
- Seems just like having one big dice roll at the beginning of the game
- Idea: compute the minimax value of each action in each deal, then choose the action with highest expected value over all deals.
- Special case: if an action is optimal for all deals, it's optimal
- GIB, current best bridge program, approximates this idea by:
  - Generating 100 deals consistent with bidding information
  - Picking the action the wins the most tricks on average

Proper Analysis

* Intuition that the value of an action is the average of its values in all actual states is WRONG

- With partial observability, value of an action depends on the information state or belief state the agent is in
- Can generate and search a tree of information states
- Leads to rational behaviors such as:
  - Acting to obtain information
  - Signaling to one's partner
  - Acting randomly to minimize information disclosure

Coming up…

- What if the game is too complex for minimax and related searches?
- Can we do reasonably well using some other approach?
  - Monte Carlo methods